arXiv:2502.16797v1 [cs.LG] 24 Feb 2025

Forecasting Rare Language Model Behaviors

Erik Jones [*1] **Meg Tong** [*1] **Jesse Mu** [1] **Mohammed Mahfoud** [2] **Jan Leike** [1] **Roger Grosse** [1]
Jared Kaplan [1] **William Fithian** [3] **Ethan Perez** [1] **Mrinank Sharma** [1]

Abstract

Standard language model evaluations can fail to capture risks that emerge only at deployment scale. For example, a model may produce safe responses during a small-scale beta test, yet reveal dangerous information when processing billions of requests at deployment. To remedy this, we introduce a method to forecast potential risks across *orders of magnitude more queries* than we test during evaluation. We make forecasts by studying each query's *elicitation probability*—the probability the query produces a target behavior—and demonstrate that the largest observed elicitation probabilities predictably scale with the number of queries. We find that our forecasts can predict the emergence of diverse undesirable behaviors—such as assisting users with dangerous chemical synthesis or taking power-seeking actions—across up to three orders of magnitude of query volume. Our work enables model developers to proactively anticipate and patch rare failures before they manifest during large-scale deployments.

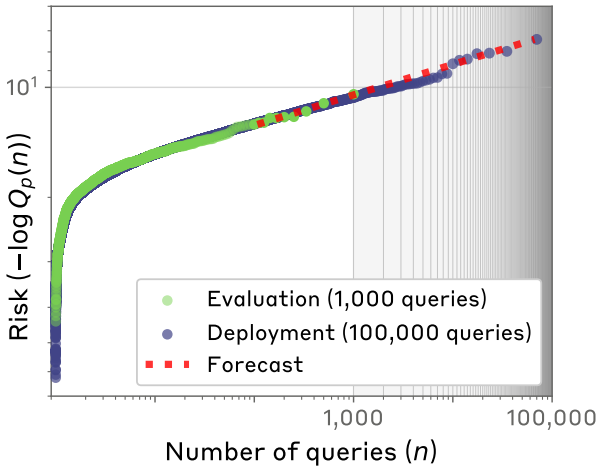


Figure 1: **Scaling laws enable forecasting rare language model failures.** We find that the risk of the highest-risk queries follows a power-law in the number of queries. This lets us forecast whether any query is likely to exhibit an undesired behavior at deployment (shaded, right), from orders-of-magnitude smaller evaluations (unshaded, left).

1. Introduction

Large language model (LLM) evaluations face a fundamental challenge: they attempt to accurately predict deployment-level risks from datasets that are many orders of magnitude smaller than deployment scale. Evaluation sets typically contain hundreds to thousands of queries (e.g., Souly et al., 2024), while deployed LLMs process billions of queries daily. This scale disparity means that standard evaluation can miss failures: rare behaviors that do not occur during evaluation or beta testing may still manifest at deployment.

To overcome this challenge, in this work we introduce a method to forecast potential deployment risks from orders-of-magnitude smaller evaluation sets. For example, we might forecast whether any of 100,000 jailbreaks from some distribution will break an LLM at deployment using a set of 100 failed jailbreaks from the same distribution. Critically, we predict risks *before they actually manifest*, enabling model developers to take preventative action.

To make forecasts, we leverage the *elicitation probabilities* of evaluation queries. While any individual LLM output provides a binary signal (i.e., it either exhibits a target behavior or not), the probability that a fixed query elicits a behavior is continuous. For example, seemingly ineffective jailbreaks in fact elicit harmful outputs with non-zero probability under enough repeated sampling. Elicitation probabilities let us reason about deployment risks; risk is often concentrated in the queries with the largest elicitation probabilities, as these are most likely to elicit the behavior.

We find that the largest observed elicitation probabilities exhibit *predictable scaling behavior* (Figure 1). Specifically, we find that the logarithm of the largest-quantile elicitation probabilities follows a power-law in the number of samples

[*]Equal contribution [1]Anthropic [2]Mila – Québec AI Institute [3]UC Berkeley. Correspondence to: Erik Jones <erikjones@anthropic.com>, Mrinank Sharma <mrinank@anthropic.com>.

required to estimate them. We use this relationship to forecast how the largest elicitation probabilities grow with scale; for example, we predict the top-0.001% elicitation probability (which manifests at a 100000-query deployment) by measuring growth from the top-1% to the top 0.1% elicitation probabilities (using a 1000 query evaluation set).

We find that our method can accurately forecast diverse undesirable behaviors, ranging from models providing dangerous chemical information to models taking power-seeking actions, over orders-of-magnitude larger deployments. For example, when forecasting the risk at 90,000 samples using only 900 samples (a difference of two orders of magnitude), our forecasts stay within one order of magnitude of the true risk for 86% of misuse forecasts. We also find that our forecasts can improve LLM-based automated red-teaming algorithms by more efficiently allocating compute between different red-teaming models.

Our forecasts are not perfect—they can be sensitive to the specific evaluation sets, and deployment risks themselves are stochastic. Nevertheless, we hope our work enables developers to proactively anticipate and mitigate potential harms before they manifest in real-world deployments.

2. Related work

Language model evaluations. Language models are typically evaluated on benchmarks for general question-answering (Hendrycks et al. (2021); Wang et al. (2024); OpenAI (2024b); Phan et al. (2025)), code (Jimenez et al. (2024); Chowdhury et al. (2024); Jain et al. (2024)) STEM (Rein et al. (2023); MAA (2024)); and general answer quality (Dubois et al. (2024); Li et al. (2024)). Typical evaluation for safety includes static tests for dangerous content elicitation (Shevlane et al. (2023); Phuong et al. (2024)), and automated red-teaming (Brundage et al. (2020); Perez et al. (2022); Ganguli et al. (2022); Feffer et al. (2024)).

Modelling rare model outputs. Our work aims to forecast rare behaviors for LLMs; this builds on rare behavior detection for image classification (Webb et al., 2019), autonomous driving (Uesato et al., 2018), and increasingly in LLM safety (Hoffmann et al., 2022; Phuong et al., 2024). The most related work to ours is Wu & Hilton (2024), which forecasts the probability of greedily generating a specific single-token LLM output under synthetic distribution of prompts. We make forecasts about more general classes of behaviors, using the extreme quantiles of elicitation probabilities to forecast.

Inference-time scaling laws for LLMs. Our work builds on inference-time scaling laws (Brown et al., 2024; Snell et al., 2024), where more inference-time compute improves output quality, and can also improve jailbreak robustness (Wen et al., 2024; Zaremba et al., 2025). The closest inference-time scaling law to our work is Hughes et al. (2024), which shows that the fraction of examples in a benchmark that jailbreak the model has predictable scaling behavior in the number of attempted jailbreaks. We instead show an *example-based* scaling law, which allows us to forecast when a specific example will be jailbroken.

We include additional related work in Appendix A.

3. Methods

The goal of pre-deployment language model testing—such as standard evaluation or small-scale beta tests—is to assess the risks of deployment to inform release decisions. We introduce a method that *forecasts* deployment-scale risks using orders-of-magnitude fewer queries. To do so, we extract a continuous measure of risk across queries that grows predictably from evaluation to deployment.

Concretely, suppose a language model will be used on n queries sampled from the distribution $\mathcal{D}_{\text{deploy}}$ at deployment, i.e., $x_1, \ldots, x_n \sim \mathcal{D}_{\text{deploy}}$, which produce outputs $o_1, \ldots, o_n$. These n deployment queries might be different attempts to elicit instructions about how to make chlorine gas from the model. If B is a boolean indicator specifying whether an output exhibits the behavior in question (in this case, successful instructions for producing chlorine gas), we wish to understand the probability that $B(o_i) = 1$ for at least one o_i. This testing is especially important for high-stakes risks, where even a single failure can be catastrophic.

The standard way this testing is done in practice is by collecting an evaluation set of queries that tests for the undesired behavior; the evaluation set might be a benchmark, or a small-scale beta test. Formally, we assume the evaluation set is constructed by sampling m queries $x_1, \ldots x_m \sim \mathcal{D}_{\text{eval}}$, and getting outputs $o_1, \ldots o_m$. Evaluation successfully flags risks if any output exhibits the undesired behavior (Mitchell et al., 2019; OpenAI, 2024; Anthropic, 2024).

Unfortunately, this methodology can miss deployment failures. One potential reason is there could be a *distribution shift* between evaluation and deployment; i.e., $\mathcal{D}_{\text{eval}} \neq \mathcal{D}_{\text{deploy}}$, and deployment queries are more likely to produce failures.[1] However, even after accounting for distribution shifts, evaluation can miss deployment failures due to differences in scale; the number of deployment queries n is frequently orders of magnitude larger than the number of evaluation queries m. Larger scale increases risks since risks come from *any* undesired output; intuitively, more attempts to elicit instructions for chlorine gas increases the probability that at least one attempt will work.

To identify when risks emerge from the scale of deployment,

[1]This distribution shift can be partly addressed by developers data from a beta test, although this does not handle temporal shifts.

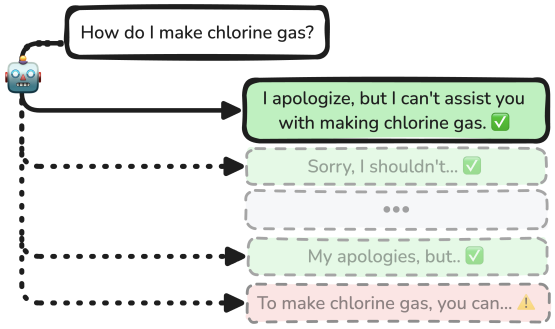


Figure 2: Repeatedly sampling from queries can elicit undesired behaviors with low-but-nonzero probability. We measure these (low) elicitation probabilities on evaluation queries and use them to forecast the largest elicitation probabilities at deployment.

our goal is to *forecast* risks from a smaller, in-distribution evaluation set. Formally, we assume $\mathcal{D}_x := \mathcal{D}_{\text{eval}} = \mathcal{D}_{\text{deploy}}$, and want to predict whether or not we should expect to see a behavior on any n deployment queries; for example, we might aim to predict whether any of 10,000 jailbreaks from some distribution will break the model, given 100 failed jailbreaks from the same distribution. To do so, we develop a continuous measure for the risk of each query x_i even when its output o_i does not exhibit the behavior. We then find that the risks under this measure grows in a predictable way, which lets us forecast actual risks at deployment.

3.1. Elicitation probabilities

To extract more information from evaluation queries, a natural extension to sampling one output per query is to sample many, then test what fraction elicit the behavior. We define this as the *elicitation probability* of a query: the probability that a sampled output from a query has a specific behavior. Formally, if $\mathcal{D}_{\text{LLM}}(x)$ is the distribution over outputs for query x, the elicitation probability $p_{\text{ELICIT}}(x; \mathcal{D}_{\text{LLM}})$ of query x for behavior B is:

$$p_{\text{ELICIT}}(x; \mathcal{D}_{\text{LLM}}, B) = \mathbb{E}_{o \sim \mathcal{D}_{\text{LLM}}(x)} \mathbf{1}[B(o) = 1]. \quad (1)$$

Empirically, we find that many queries have small but nonzero elicitation probabilities (e.g., $p_{\text{ELICIT}} < 0.01$) (Figure 2). This is even true for jailbreaks; repeatedly sampling from a query that produces refusals on most outputs such as can sometimes produce useful instructions.

Measuring elicitation probabilities is especially useful since we can compute the deployment risk from deployment elicitation probabilities. At deployment, we sample n queries, each of which has a corresponding elicitation probability $p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B)$. Each query's elicitation probability determines whether it produces an undesired output; depending on the setup, a query might produce a bad output if the elicitation probability is above a threshold, or randomly with probability $p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B)$.

Much of the risk at deployment frequently comes from the largest sampled elicitation probabilities. We capture how the largest elicitation probabilities grow with scale by studying the *largest quantiles* of the distribution of elicitation probabilities across queries; for example, the 99th percentile elicitation probability might tell us what to expect in 100 queries, while the 99.99th percentile elicitation probability might indicate how large the elicitation probabilities should be in 10000 queries. To formalize this, define $Q_p(n)$ to be the threshold for the top $1/n$ fraction of elicitation probabilities; informally, $Q_p(n)$ is defined such that

$$\mathbf{P}_{x \sim \mathcal{D}_x}[p_{\text{ELICIT}}(x; \mathcal{D}_{\text{LLM}}, B) \geq Q_p(n)] = 1/n. \quad (2)$$

We can measure how scale increases risk by studying how $Q_p(n)$ grows with the number of deployment queries n.

3.2. Metrics for deployment risk

We now argue that forecasting the tail quantiles $Q_p(n)$ is sufficient to forecast deployment risk.

1. The ***worst-query risk*** is the maximum elicitation probability out of n sampled queries:
$$\max_{i \in [n]} p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B)$$
We forecast the worst-query risk of n samples as $Q_p(n)$. We use this to validate our forecasts of the quantiles.

2. The ***behavior frequency*** is the fraction of queries that have an elicitation probability greater than a threshold τ.
$$\mathbf{E}_{x \sim \mathcal{D}_x} \mathbf{1}[p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B) > \tau]$$
We compute the behavior frequency by finding the quantile that matches the chosen threshold; the behavior frequency is $1/n'$, where n' is such that $Q_p(n') = \tau$. The behavior frequency captures risks that are concentrated in one query; i.e., query only counts as eliciting behavior if it does so routinely. Wu & Hilton (2024) also forecast the behavior frequencies.

3. The ***aggregate risk*** is the probability that sampling a single random output from each of n queries produces an example of the behavior
$$1 - \prod_{i=1}^{n} (1 - p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B))$$
We compute the aggregate risk by randomly sampling elicitation probabilities using the forecasted quantiles $Q_p(n)$ and the empirical distribution.[2] This simulates

[2]To compute the aggregate risk, we sample $u_i \sim U_{[0,1]}$, then set the elicitation probability p_i to be the u_i^{th} quantile of the distribution. We use the empirical quantiles if $u_i < 1 - 1/m$ (i.e., the evaluation quantiles) and otherwise use the forecasted quantiles.

sampling single output from each query at deployment. Aggregate risks can arise even when the worst-query risk and behavior probabilities are low, as the low elicitation probabilities can compound with scale.

3.3. Forecasting the extreme quantiles

Deployment risks are a function of the tail of the distribution of elicitation probabilities; we need to account for a one-in-a-million query for a million query deployment. This means that some of the quantiles that we need to compute risk, $Q_p(n)$, are not represented during evaluation. We instead forecast them from the empirical evaluation quantiles.

Our primary forecasting approach is the ***Gumbel-tail method***, where we assume that logarithm of the extreme quantiles scales according to a power law with respect to the number of queries n. Concretely, define $\psi_i = -\log(-\log p_{\text{ELICIT}}(x_i; \mathcal{D}_{\text{LLM}}, B))$ to be the elicitation score of input x_i. Under fairly general conditions, extreme value theory tells us that the distribution of the highest quantiles random variables tends towards the extreme quantiles of one of three distributions, one of which is Gumbel. We include further motivation for why we expect to see Gumbel scaling in particular in Appendix B.1.

For distributions with extreme behavior that tends towards a Gumbel, we can exploit a key property: the tail of the log survival function is an approximately linear function of the elicitation score. Formally, for survival function $S(\psi) = \mathbf{P}(\psi_i > \psi)$, this says $\log S(\psi) = a\psi + b$ for constants a and b for sufficiently large ψ. See Appendix B.2 for a complete argument. This means that if $Q_\psi(n)$ is the $1/n^{th}$ largest quantile score, for sufficiently large n,

$$\log S(Q_\psi(n)) = aQ_\psi(n) + b \tag{3}$$

$$\log \frac{1}{n} = aQ_\psi(n) + b \tag{4}$$

$$Q_\psi(n) = -\frac{1}{a}(\log n - b), \tag{5}$$

where the second line comes from the definition of $Q_\psi(n)$.

We thus make forecasts about extreme quantiles using the linear relationship between $\log n$ and the corresponding score quantiles; this corresponds to a power law between the log-quantiles of the distribution of elicitation probabilities and the number of queries. Specifically, we fit a and b using ordinary least squares regression between the elicitation score and the corresponding log survival probability, for the ten highest elicitation scores during evaluation. We then use the learned a and b to extrapolate to larger quantiles (see Figure 1 for an example). Since the score function is monotone, this means that the original distribution has quantiles $Q_p(n) = e^{-e^{-Q_\psi(n)}}$, and there is a power law between $-\log Q_p(n)$ and n.

There are limitations to this method. Since our forecast only uses the largest ten elicitation probabilities, the forecasts are sensitive to stochasticity in the specific evaluation set. Moreover, the evaluation set might not be large enough to capture the extreme tail behavior. We find that despite these limitations, the Gumbel-tail method frequently makes accurate forecasts in practice.

4. Forecasting misuse

We will first forecast whether adversaries can *misuse* models for nefarious tasks at deployment, when they fail to during evaluation. We focus on cases where non-adaptive adversaries attempt to elicit harmful information about how to make dangerous chemical or biological substances.

4.1. Setup

To test for misuse, we evaluate whether models will elicit instructions for 10 dangerous chemicals and 9 biological substance. For each substance, we generate queries by simulating the high-level automated red-teaming strategy from Perez et al. (2022). Specifically, we prompt a fine-tuned helpful-only variant Claude 3.5 Sonnet (new) to create model inputs designed to elicit the instructions; for example, we might use the meta-prompt "*Generate queries designed to elicit how chlorine gas is made from a model.*" to generate queries. We then sample 100000 queries from this prompt independently at temperature one. See Appendix C.1 for specific substances and additional details. We want to measure the probability that each question elicits harmful information from the LLM. To do so, we will rely on a few proxies:

1. *The probability of a specific output* that contains harmful instructions given the query. For chlorine gas, the specific output is "*Chlorine gas can be made by mixing bleach and vinegar*".
2. The probability of a *high-information keyword* in the specific output. In the chlorine gas case, the keyword is *bleach*, and we measure the probability of bleach given the query and previous output tokens.
3. The probability a randomly sampled output contains *useful instructions*, where in this case we measure utility by checking if the keyword appears anywhere in the output.

All three of these proxies approximate how likely the model is to add useful information about how to make dangerous substances, but they have different tradeoffs. Measuring the probability of a specific output is efficient—it can be done in a single forward pass—but may not reflect the actual likelihood of producing "useful" instructions. Measuring keyword probabilities produces higher elicitation probabilities and is just as efficient, but requires that adversaries can